
Learning by Remembering

Contact Session 7 — Module 6

Some methods study hard and boil everything down to a formula. These ones do the opposite: they just remember every example and look up the closest ones when asked. Simple, and surprisingly powerful.

- | | |
|------------------|------------------|
| ■ Intuition | ■ Analogy |
| ■ Worked example | ■ Solved problem |
| ■ Common pitfall | ■ Key takeaways |

Contents

1	Two ways to be smart	3
2	Nearest neighbours: copy your closest look-alikes	4
2.1	How many neighbours? The k dial	4
3	The one big weakness: too many facts	6
4	A smarter cousin: fit a tiny model near the question	7
5	Building shapes out of bumps (radial basis functions)	8
6	When to use these, and when not	9
7	The whole story on one page	10

1 Two ways to be smart

Every method so far did its hard work *up front*: it chewed through the data, boiled it down to a compact model (some weights, a tree), and then threw the data away. We call that **eager** learning. This session is about the opposite idea. A **lazy** learner does *no* work up front. It simply **remembers all the examples**. When you ask it about a new case, only *then* does it spring into action: it finds the remembered examples most similar to your case and answers based on them.

Eager (study hard)

does all the work while training;
quick to answer later
regression, trees, logistic

Lazy (just remember)

no work while training;
does the work when asked
nearest neighbours, and friends

↪ The crammer and the open-book student

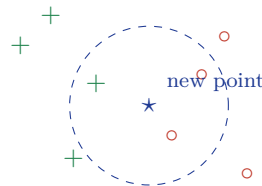
The eager learner is the student who studies all term and walks into the exam with everything compressed in their head — quick to answer, nothing to carry. The lazy learner studies nothing but brings the whole textbook; for each question they flip to the most relevant page and answer on the spot. Remembering is instant; the *answering* is where the effort goes.

The one bet behind all of this: **similar things have similar answers**. If a new house looks just like three houses that sold for around 60 lakh, it's probably worth around 60 lakh too. No grand formula — just “find the look-alikes and copy them.”

2 Nearest neighbours: copy your closest look-alikes

The simplest lazy method is ***k*-nearest neighbours** (kNN). To answer a question about a new point:

1. Measure how far the new point is from *every* remembered example.
2. Keep the *k* closest ones (its *k* “neighbours”).
3. For a group answer, take a **vote** of those neighbours. For a number answer, take their **average**.



the 3 nearest neighbours (inside the circle)

“How far” usually means plain straight-line distance — the same distance you’d measure with a ruler. For two facts it’s the school formula, and for many facts you just keep adding squares under the root:

$$d = \sqrt{(\Delta\text{fact}_1)^2 + (\Delta\text{fact}_2)^2 + \dots},$$

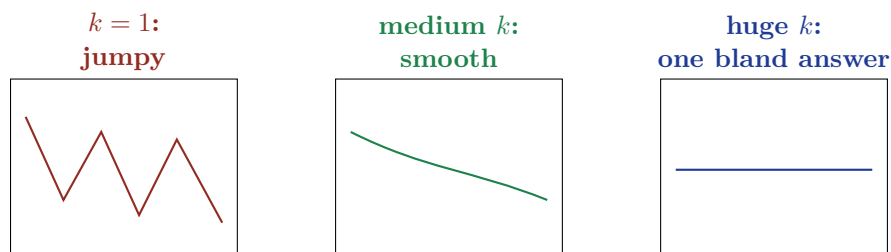
where Δfact_1 means “the difference in fact 1 between the two points.” Squaring keeps every difference positive, and the square-root brings the answer back to normal units. (A common alternative is “city-block” distance — just add up the plain differences $|\Delta\text{fact}_1| + |\Delta\text{fact}_2| + \dots$, the way you walk a grid of streets. Both are fine; straight-line is the default.)

! Scale your facts first — this one really bites

Distance is bossed around by big-numbered facts. If *income* (in lakhs) and *age* (in years) are both facts, income’s size swamps everything and age is ignored entirely — two people of very different ages but similar income look “identical.” So you *must* shrink every fact to a similar size first (Module 2). For nearest neighbours this isn’t optional.

2.1 How many neighbours? The *k* dial

k is a dial you turn, and it trades two kinds of mistake.



- $k = 1$ (trust only the single closest): super flexible, but one weird neighbour can flip the answer — jumpy.
- **Big k** (average many): smooth and steady, but if k is too big you're just averaging half the dataset and ignoring *where* the question even was.
- **In practice:** try a few values and keep the one that does best on held-aside data; an odd k avoids tie votes.

A nice tweak: let *closer* neighbours count for more. A neighbour right next to your point should have a louder vote than one at the edge of the circle — so we weight each vote by closeness. Sensible, and it usually helps.

Σ Vote with $k = 3$

New point at $(5, 6)$. Five remembered points, with their straight-line distances worked out:

point	where it is	label	distance to $(5, 6)$
A	$(4, 7)$	$+$	$\sqrt{1+1} = 1.41$
B	$(6, 6)$	$-$	$\sqrt{1+0} = 1.00$
C	$(5, 4)$	$-$	$\sqrt{0+4} = 2.00$
D	$(3, 5)$	$+$	$\sqrt{4+1} = 2.24$
E	$(7, 7)$	$-$	$\sqrt{4+1} = 2.24$

The three closest are B (1.00, $-$), A (1.41, $+$) and C (2.00, $-$). Two say “ $-$,” one says “ $+$,” so the answer is $-$. (Weighting by closeness would lean even harder on B , the nearest — and here it agrees.)

3 The one big weakness: too many facts

Nearest neighbours has a famous Achilles heel. When each example has *lots* of facts, “distance” stops meaning much, because everything ends up roughly equally far from everything else.

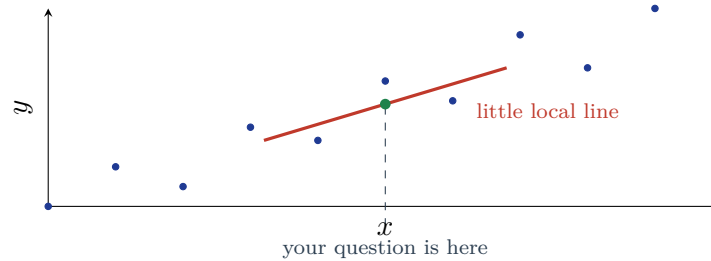
↪ Lost in a huge space

On a 100-metre line, your ten nearest neighbours really are close by. On a football field, less so. In a giant warehouse, less still. In a space with a hundred facts, the room is so vast that even your “nearest” neighbour is miles away — and barely closer than the farthest. The whole idea of “nearby look-alikes” quietly falls apart.

The fixes are the feature moves from Module 2: drop useless facts, or squash many facts down to a few important ones (PCA). Fewer, better facts bring the neighbourhood back to life.

4 A smarter cousin: fit a tiny model near the question

Plain nearest-neighbours just averages the neighbours. A neater idea, **locally weighted regression**, is: gather the points *near* your question, draw a little straight line through just those (letting closer points pull harder), and read your answer off that little line.



“Letting closer points pull harder” has a simple formula. A point sitting at distance d from your question gets a weight

$$\text{weight} = e^{-d^2/2\tau^2}.$$

Read it gently: a point right on top of the question ($d = 0$) gets weight $e^0 = 1$ (full say), and the weight fades toward 0 as d grows. The knob τ sets *how fast* it fades — a small τ listens only to the very nearest points, a big τ listens to a wide crowd. (It’s the same bell-shaped fade we’ll reuse for the bumps below.)

★ A fresh answer for every question

Instead of one big rule for the whole world, it builds a tiny custom rule right where you asked, using mostly the nearby data. Ask somewhere else and it builds a fresh little rule there. Stitch all those local lines together and you get a smooth curve that can follow almost any shape — without ever committing to a single global equation.

5 Building shapes out of bumps (radial basis functions)

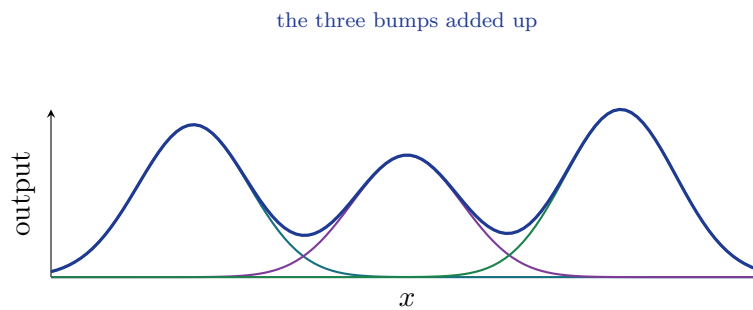
One more idea worth meeting. A **radial basis function** model builds its answer by stacking up little *bumps*. Each bump is tall near its own centre and fades to nothing far away — and it's the *same* bell-shaped fade as the weight above. A bump centred at a point c has value

$$\text{bump}(x) = e^{-d^2/2\sigma^2}, \quad d = \text{distance from } x \text{ to the centre } c,$$

so it equals 1 right at the centre ($d = 0$) and fades to 0 far away; σ sets how wide it spreads. The final answer is just these bumps, each scaled by a tunable height w_k , added up:

$$\text{answer}(x) = w_1 \cdot \text{bump}_1(x) + w_2 \cdot \text{bump}_2(x) + \dots$$

Tune the heights and you can shape almost any curve.



↪ Spotlights on a dark stage

Picture a few spotlights aimed at a stage; each lights a soft circle, and you can dial each one's brightness. By tuning the brightnesses you can light up almost any pattern across the stage. The bumps are the spotlights, their heights are the brightnesses, and the lit-up pattern is the model's answer. (Where to point the lights is usually decided by clustering the data first.)

6 When to use these, and when not

Lovely because	Annoying because
nothing to train — just store the data	slow to answer (must scan everything)
can fit very wiggly, complicated shapes	breaks down with too many facts
add new data anytime, instantly	must keep all the data around
no assumptions about the world	easily fooled by unscaled or noisy facts

★ Key Takeaways

Reach for “learn by remembering” when the pattern is wiggly, you have plenty of data with not-too-many facts, and you need training to be instant. Avoid it when there are very many facts, or when answers must be lightning-fast on a giant dataset (then an eager model, or a clever index to speed up the neighbour search, is better).

7 The whole story on one page

★ Key Takeaways

- **Lazy** learning just remembers the examples and does the work when asked, betting that *similar things have similar answers*.
- **Nearest neighbours**: find the k closest remembered points and **vote** (groups) or **average** (numbers). You must **scale your facts** first.
- k is a dial: small = jumpy, big = bland. Tune it; let closer neighbours vote louder.
- Watch the **too-many-facts** trap — distance stops meaning anything; fix by dropping or squashing facts.
- **Locally weighted regression** fits a tiny local line per question for smoother answers.
- **Radial basis functions** build answers by stacking tunable **bumps**.
- Trade-off: no training at all, very flexible — but slow to answer, hungry for memory, and easily thrown off by too many or unscaled facts.

Where this comes from: Mitchell, *Machine Learning*, Ch. 8. Course syllabus Module 6 (BITS Pilani WILP).